Enterprise-Knowledge-Assistant

1. Project Goal

Build an enterprise-ready AI knowledge copilot that allows employees to ask natural-language questions about organizational documents and receive grounded, contextual answers. The system combines document ingestion, retrieval-augmented generation (RAG), persistent conversation memory, safety controls, human approval for sensitive requests, and execution observability.

2. Problem Statement

Organizational knowledge is scattered across policies, handbooks, procedures, and internal documents. Employees waste time searching for answers, responses can be inconsistent, and ordinary chatbots may hallucinate, lack access to current documents, or provide insufficient controls for sensitive operations. Teams also need visibility into failures, response latency, and the steps taken by AI workflows.

3. Proposed Solution

Enterprise-Knowledge-Assistant provides a single chat interface over approved enterprise documents. Files are uploaded to Amazon S3, processed into searchable chunks, embedded with Amazon Bedrock Titan, and indexed in Pinecone. A LangGraph workflow retrieves relevant context, uses Amazon Nova Lite to compose an answer, and applies guardrails. PostgreSQL stores chat history and application records; potentially sensitive actions enter a human-in-the-loop (HITL) approval queue. GlassFlow Rius provides observability for instrumented operations.

4. Real-World Use Cases

HR and employee policies: Ask about leave entitlement, working hours, remote work, and onboarding rules.

IT and security knowledge: Find password, MFA, device security, and incident-reporting procedures.

Operations and travel policies: Retrieve reimbursement rules, approval chains, and internal process guidance.

Multi-turn knowledge assistance: Ask follow-up questions without repeating the original context.

Controlled administrative requests: Route document deletion or policy modification requests for human review before any separately implemented execution.

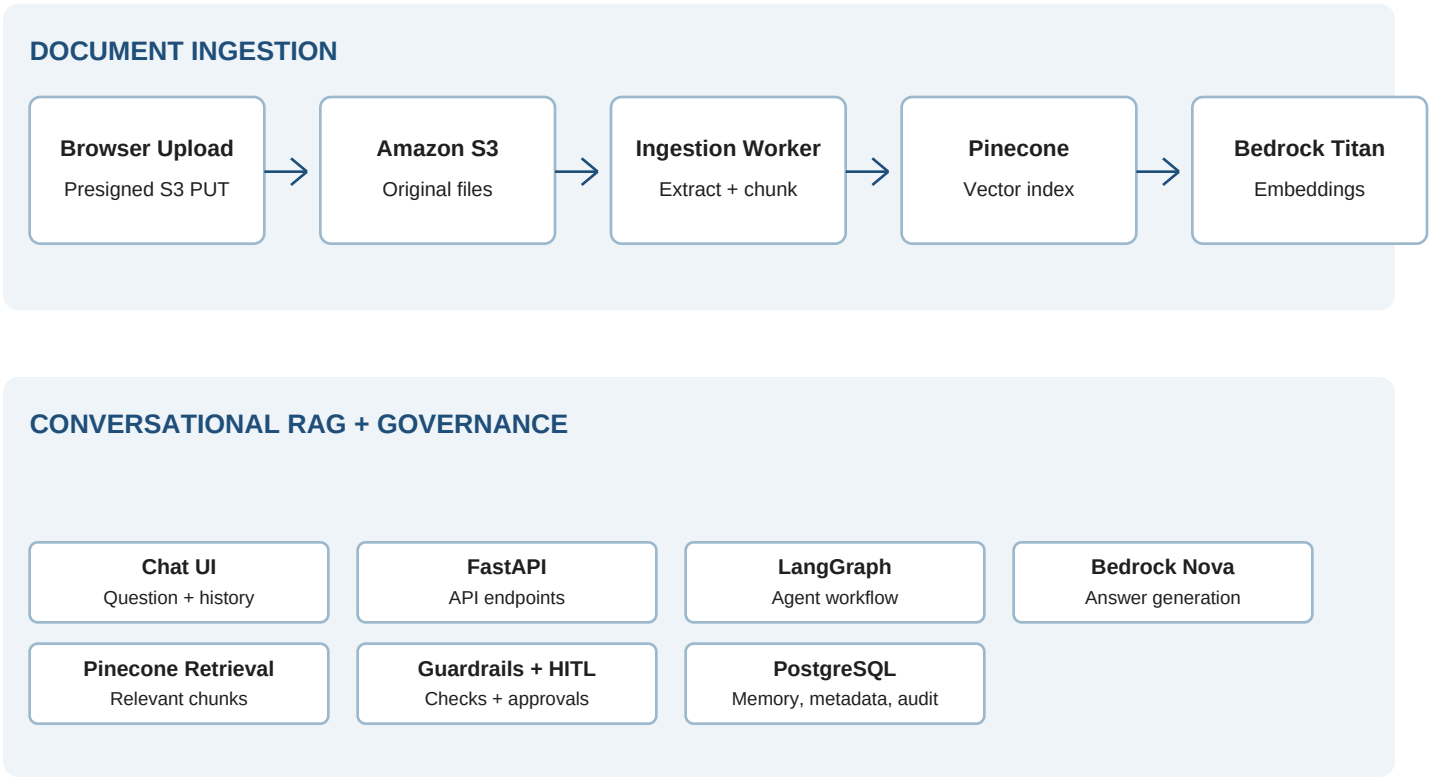
Technology Stack & Responsibilities

Each component has a defined role in the system.

Technology	Purpose
Python + FastAPI	Backend APIs for chat, uploads, ingestion triggers, approvals, and metrics.
HTML, CSS, JavaScript	Browser dashboard with chat, upload, approvals, and scrollable history.
LangGraph	Orchestrates retrieval, memory, guardrails, model calls, and HITL routing.
Amazon S3	Durable document storage; browser uploads via presigned URLs.
Amazon Bedrock Titan Embeddings V2	Converts document chunks and queries into semantic vectors (1024 dimensions).
Amazon Bedrock Nova Lite	Generates responses using retrieved enterprise context.
Pinecone	Stores embeddings and retrieves semantically similar chunks.
Render PostgreSQL	Persists document metadata, conversations, approvals, feedback, and audit events.
GlassFlow Rius	Traces instrumented agent, ingestion, model, and retrieval operations.
Deterministic Python guardrails	Screens selected input/output patterns; not a substitute for full security controls.
Ingestion worker (worker/ingest.py)	Extracts text, chunks documents, creates embeddings, and updates indexes/status.

System Architecture

Two primary paths: document ingestion and conversational question answering.



How the Architecture Works

Ingestion: A user uploads a document to S3. The ingestion worker extracts and chunks text, requests Titan embeddings, stores vectors in Pinecone, and updates document metadata/status in PostgreSQL.

Chat: A user question reaches FastAPI and LangGraph. The workflow consults conversation history, retrieves relevant Pinecone chunks, invokes Nova Lite, applies guardrails, and returns a grounded response. Sensitive requests can create HITL approval records instead of immediately performing the requested action.

Observability: GlassFlow Rius receives traces from explicitly instrumented operations. Tracing should avoid exporting sensitive document or prompt content by default.

Application Functionalities

What learners can demonstrate in Enterprise-Knowledge-Assistant

Functionality	Description
Document upload & ingestion	Upload PDF/TXT and supported documents directly to S3 using presigned URLs. The ingestion worker extracts text, chunks it, generates Titan embeddings, indexes Pinecone vectors, and tracks status.
Enterprise RAG chat	Ask natural-language questions. LangGraph retrieves relevant Pinecone context and uses Bedrock Nova Lite to generate document-grounded responses, with source references where available.
Persistent conversation memory	Store chat messages in PostgreSQL and use recent conversation turns to understand follow-up questions.
Input & output guardrails	Apply deterministic checks to detect selected prompt-injection patterns, requests for secrets, and potentially unsafe generated output. These are application-level controls.
Human-in-the-loop (HITL)	Route sensitive requests such as document deletion or policy modification to a pending approval queue. Administrators can approve or reject requests in the dashboard.
Audit trail & feedback	Record significant application events and approval decisions in PostgreSQL; capture user feedback for review and improvement.
Observability with GlassFlow Rius	Instrument LangGraph nodes, document ingestion, Pinecone operations, and Bedrock calls to inspect execution traces, latency, and errors. Sensitive-content capture can be disabled.
Dashboard & usability	Provide chat, document upload, approvals, and metrics views; keep chat at a fixed height with scrollable message history and auto-scroll to new messages.
Tenant-aware data model	Associate records with tenant/workspace identifiers where implemented. Production-grade tenant isolation still requires authentication, authorization, and enforcement.

Implementation scope: HITL approval currently records decisions; approved deletion or policy changes are not automatically executed. Observability depends on configured instrumentation and a working Rius connection.